1. Beadandó feladat dokumentáció

Készítette:

Ács Gergő

E-mail: jkgvs8@inf.elte.hu

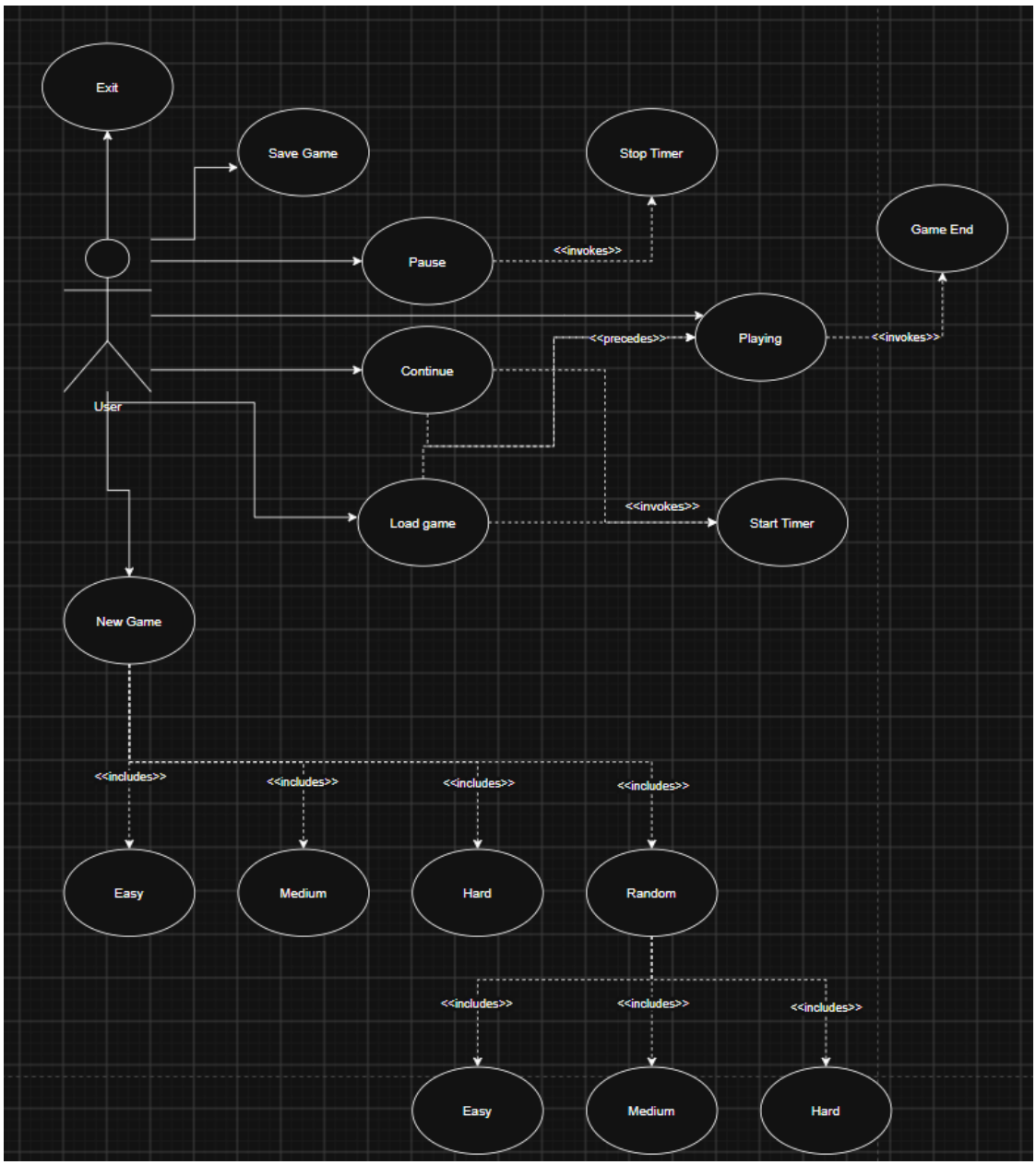
Feladat:

Készítsünk programot, amellyel a nonogram (más néven Hanjie vagy Griddlers) japán képrejtvényt oldhatjuk meg. Adott egy n × n mezőből álló tábla, amely egy rejtett fekete-fehér képet tartalmaz. Minden sor bal oldalán és minden oszlop fölött egy számsor áll, amely sorrendben megadja, hogy az adott sorban vagy oszlopban milyen hosszú, összefüggő fekete szakaszok találhatók. A szakaszok között legalább egy fehér mező van. A játékos célja, hogy a számsorok alapján megtalálja a kép összes fekete mezőjét. A játékos egy mezőt kitölthet (feketének jelöl), vagy kizárhat (fehérnek jelöl, Xszel). Egérrel a bal gomb kitölt, a jobb gomb kizár. Húzással egy sorban vagy oszlopban több mező is jelölhető egyszerre. A kitöltést a program azonnal összeveti a képpel. Ha a mező a képen fehér, a kitöltés hibás: a mező nem lesz fekete, hanem automatikusan kizárt lesz, és a játékos egy életet veszít. A játékosnak 3 élete van. A kizárás csak a játékos jelölése, ezt a program nem ellenőrzi. Ha egy sor vagy oszlop összes fekete mezőjét megtalálta a játékos, a program halványítsa el az ahhoz tartozó számsort, és a sor vagy oszlop maradék mezőit jelölje kizártnak. A képeket fájlban tároljuk, a program méretenként legalább 3 képet tartalmazzon. Ezen felül legyen lehetőség adott kitöltési arányú véletlenszerű kép előállítására is. A program biztosítson lehetőséget új játék kezdésére a táblaméret (5 × 5, 10 × 10, 15 × 15) és a kép megadásával, valamint játék szüneteltetésére (ekkor nem telik az idő, és a tábla nem látható). Ismerje fel, ha vége a játéknak: a játékos győz, ha minden fekete mezőt megtalált, és veszít, ha elfogytak az életei. A program játék közben folyamatosan jelezze ki a játékidőt és az életek számát. Ezen felül legyen lehetőség a játék elmentésére, valamint betöltésére.

Elemzés:

- A játékot három nehézségi szinttel játszhatjuk: könnyő(5X5), közepes(10X10), nehéz(15X15). Ezen felül van egy random funkció is, ami a 3 fajta pálya közül választ ki egyet random, és nehézségtől függően generál egy pályát megfelelő sűrűséggel (30%, 50%, 70%). A program indítási könnyű nehézséget állít be, és automatikusan új játékot indít.
- A feladatot egyablakos asztali alkalmazásként Windows Forms grafikus felülettel valósítjuk meg.
- Az ablakban elhelyezünk egy menüt a következő menüpontokkal: New Game (easy, medium, hard, random), Pause/Continue, Load Game, Save Game, Exit. Az ablak alján megjelenítünk egy státuszsort, amely az időt, és az életek számát jeleníti meg.

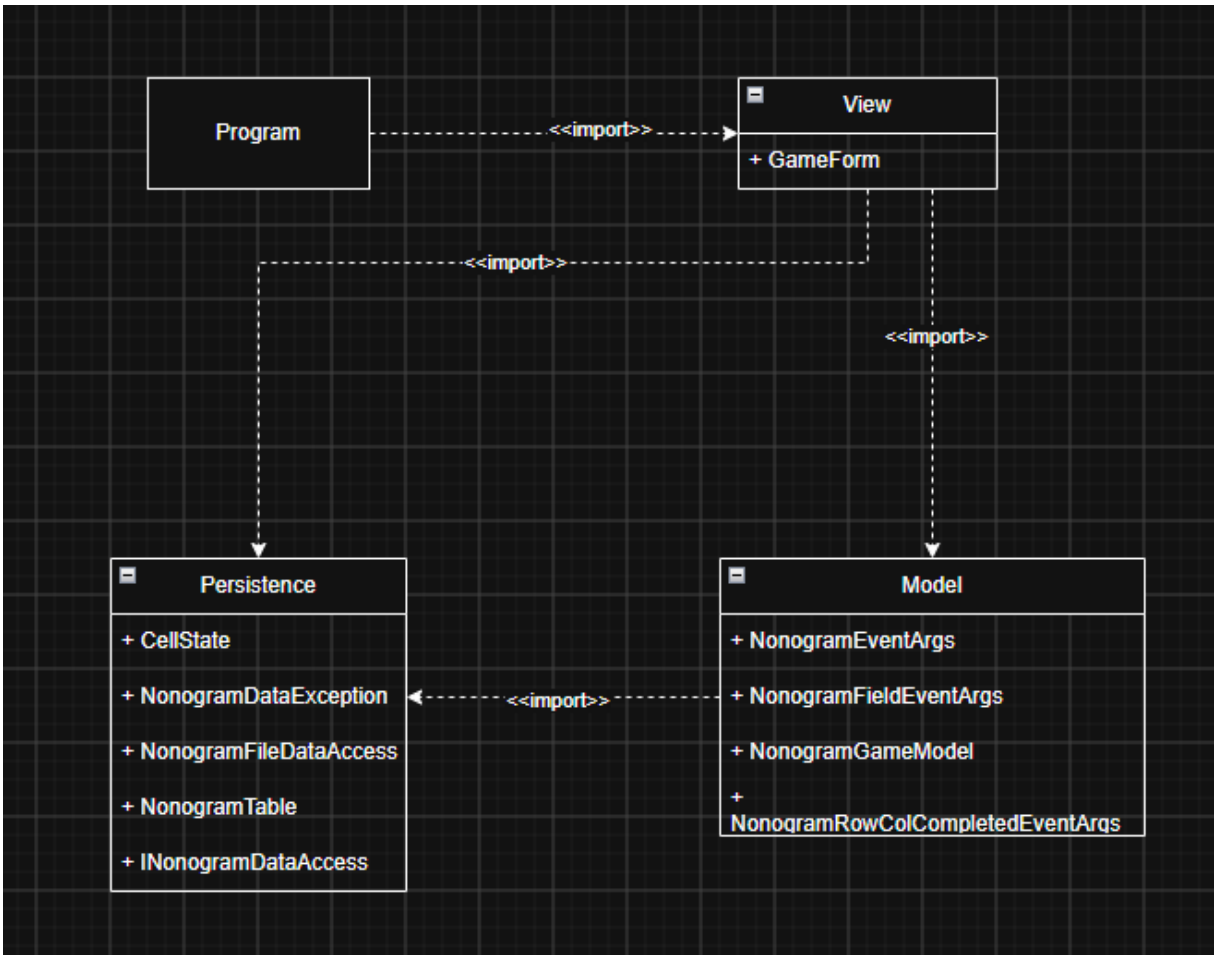
- A játéktáblát egy pálya mérettől függő nyomógombokból álló rács reprezentálja. A nyomógomb egérkattintás hatására megváltoztatja annak állapotát. Az állapot alapvetően üres, ha bal egérgombot nyom, kitöltött lesz, ha jobb egérgombot nyom, kizárt lesz. Ha kitölt egy olyan mezőt amit nem kellene, azonnal kizárt lesz, és veszít egy életet.
- A játék automatikusan feldob egy dialógusablakot, amikor vége a játéknak (kiraktuk a mintát, vagy elfogytak az életek). A mentést és betöltést fájlkezelő segítségével oldjuk meg, mentéskor a felhasználó adja meg a file nevét.
- A játék mentésekor és betöltésekor gondoskodni kell az eltelt idő és a megtett lépésszám tárolására és visszaállítására is.
- A felhasználói esetek az 1. ábrán láthatóak.



1. ábra

Tervezés:

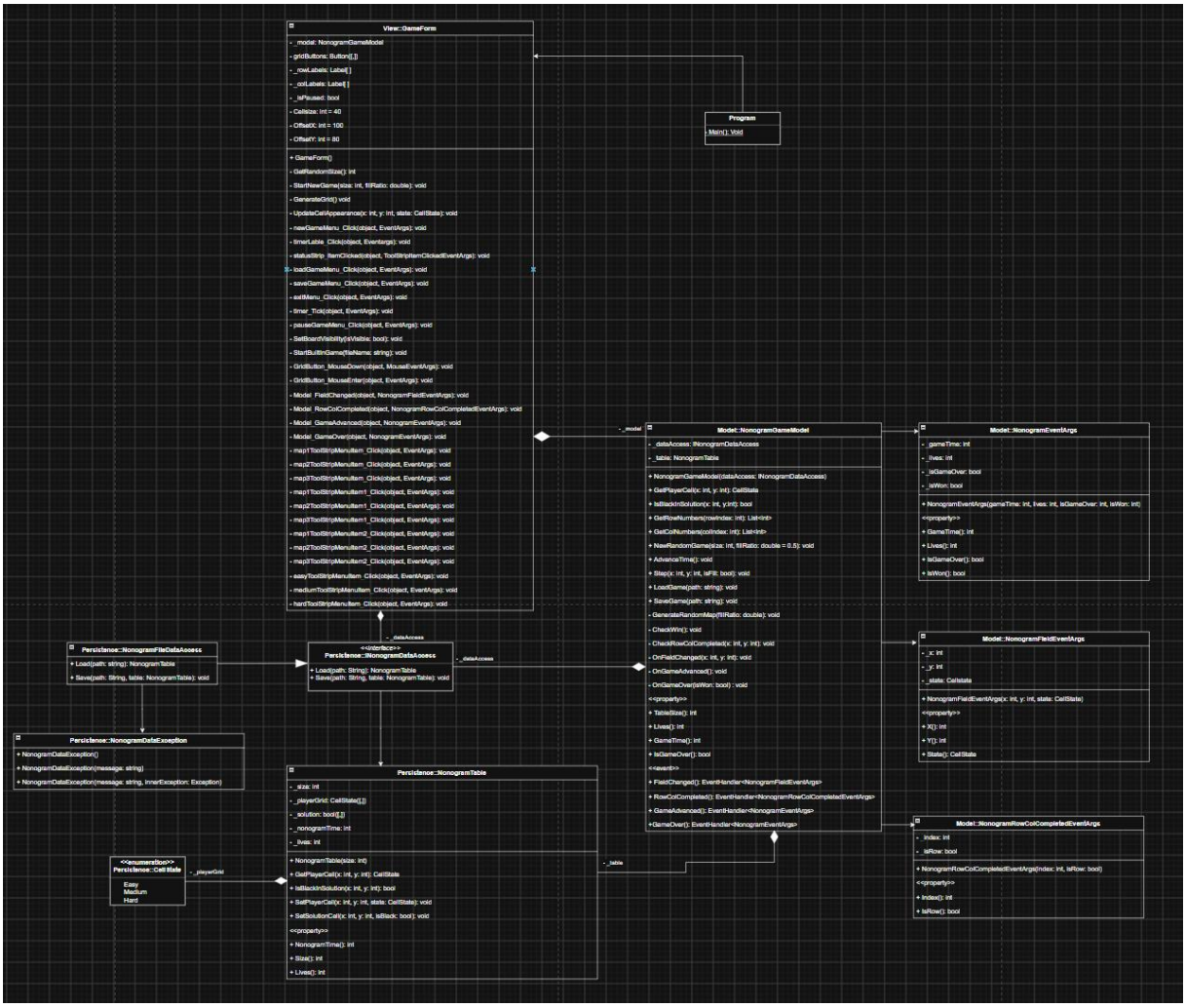
- Programszerkezete:
  - A programot háromrétegű architektúrában valósítjuk meg. A megjelenítés a **View**, a modell a **Model**, míg a perzisztencia a **Persistence** névtérben helyezkedik el. A program csomagszerkezete a 2. ábrán látható.
  - A program szerkezetét két projektre osztjuk implemetációs megfontolásból: a **Persistence** és **Model** csomagok a program felületfüggetlen projektjében, míg a **View** csomag a Windows Formstól függő projektjében kap helyet.



2. ábra

- Perzisztencia:
  - Az adatkezelés feladata a Nonogram táblával kapcsolatos információk tárolása, valamint a betöltés/mentés biztosítása.
  - A CellState enum osztályban tároljuk a lehetséges mező állapotokat (**Empty**, **Filled**, **Crossed**).
  - A **NonogramTable** osztály egy érvényes Nonogram táblát biztosít (azaz mindig ellenőrzi a beállított értékek), ahol minden mezőre ismert az értéke (**_playerGrid**). A játék kezdetekor generáljuk le az értékeket a minta alapján, CellState enum segítségével. A tábla lehetőséget az állapotok lekérdezésére

(**GetPlayerCell**, **IsBlackInSolution**), valamint szabályos léptetésre (**SetPlayerCell**), illetve direkt beállítás (**SetSolutionCell**) elvégzésére.
    - A játéktábla tárolja az eltelt időt (**_nonogramTime**), és a hátralevő életeket (**_lives**).
    - A hosszú távú adattárolás lehetőségeit az **INonogramDataAccess** interfész adja meg, amely lehetőséget ad a tábla betöltésére (**Load**), valamint mentésére (**Save**).
    - Az interfészt szöveges fájl alapú adatkezelésre a **NonogramFileDataAccess** osztály valósítja meg. A fájlkezelés során fellépő hibákat a **NonogramDataException** kivétel jelzi.
    - A program az adatokat szöveges fájlként tudja eltárolni, melyek az **txt** kiterjesztést kapják. Ezeket az adatokat a programban bármikor be lehet tölteni, illetve ki lehet menteni az aktuális állást.
    - A fájl első sora megadja a tábla méretét. Ezt követi a megoldás a pálya méretétől függően a következő x sorban. Ezt követi egy „---„ sor, ami után eltároljuk az eltelt időt, és a hátralévő életeket egy sorban. Végül a következő X sorban van a játékos általi mentés, ami alapértelmezetten egy teljesen üres pálya.
- Modell:
    - A modell lényegi részét a **NonogramGameModel** osztály valósítja meg, amely szabályozza a tábla tevékenységeit, valamint a játék egyéb paramétereit, úgy mint az idő (**_table**.**NonogramTime**) és az életeket (**_table**.**Lives**). A típus lehetőséget ad új játék kezdésére (**NewRandomGame**), valamint lépésre (**Step**).
    - A játék időbeli kezelését egy függvény végzi (**AdvanceTime**), egy eventen keresztül (**OnGameAdvance**), amelyet megállítunk és újraindítunk amikor megállítjuk a játékot, vagy újraindítjuk.
    - A mezők állapotváltozásáról a **FieldChanged** esemény tájékoztat. Az esemény argumentuma (**NonogramFieldEventArgs**) tárolja a megváltozott mező pozícióját.
    - A játékállapot megváltozásáról (életek száma, eltelt idő) a **GameAdvanced** esemény, míg a játék végéről a **GameOver** esemény tájékoztat. Az események argumentuma (**NonogramEventArgs**) tárolja a győzelem állapotát, az életek számát, valamint a játékidőt.
    - A modell példányosításkor megkapja az adatkezelés felületét, amelynek segítségével lehetőséget ad betöltésre (**LoadGame**) és mentésre (**SaveGame**).
- View:
    - A nézetet a **GameForm** osztály biztosítja, amely tárolja a modell egy példányát (**_model**), valamint az adatelérés konkrét példányát (**_dataAccess**).
    - A játéktáblát egy dinamikusan létrehozott gombmező (**_gridButtons**) reprezentálja. A felületen létrehozzuk a megfelelő menüpontokat, illetve státuszsort, valamint dialógusablakokat, és a hozzájuk tartozó eseménykezelőket. A játéktábla generálását (**GenerateGrid**), illetve az értékek beállítását (**UpdateCellAppearance**) külön metódusok végzik.
- A program teljes statikus szerkezete a 3. ábrán látható.



3. ábra

Tesztelés:

- A modell funkcionalitása egységtesztek segítségével lett ellenőrizve a **NonogramGameModelTest** osztályban.
- Az alábbi tesztesetek kerültek megvalósításra:
  - **Initialize**,
    NonogramGameModel_NewRandomGame_5x5_Test,
    NonogramGameModel_NewRandomGame_10x10_Test,
    NonogramGameModel_NewRandomGame_15x15_Test: Új játék indítása, Pálya létrehozása, idő beállítása, életek beállítása, játék végső kondíciójának ellenőrzése történik a pálya méretének függvényében.
  - **NonogramGameModel_Step_Test**: Játékbeli lépés hatásainak ellenőrzése, játék megkezdése előtt, valamint után. Több lépés végrehajtása azonos játékmezőn, esemény kiváltásának ellenőrzése.
  - **NonogramGameModel_AdvanceTime_Test**: A játékbeli idő kezelésének ellenőrzése.
  - **NonogramGameModel_LoadTest**: A játék modell betöltésének tesztelése.
  - **NonogramGameModel_SaveTest**: A játék modell mentésének tesztelése.